

HOMEWORK 5

>>NAME HERE<<

>>ID HERE<<

Instructions:

Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. Remember to assign the pages to problems when submitting.

1 Optimal Policy for Simple MDP

Consider the simple n -state MDP shown in Figure 1. Starting from state s_1 , the agent can move to the right (a_0) or left (a_1) from any state s_i . Actions are deterministic and always succeed (e.g. going left from state s_2 goes to state s_1 , and going left from state s_1 transitions to itself). Rewards are given upon taking an action from the state. Taking any action from the goal state G earns a reward of $r = +1$ and the agent stays in state G . Otherwise, each move has zero reward ($r = 0$). Assume a discount factor $\gamma < 1$.

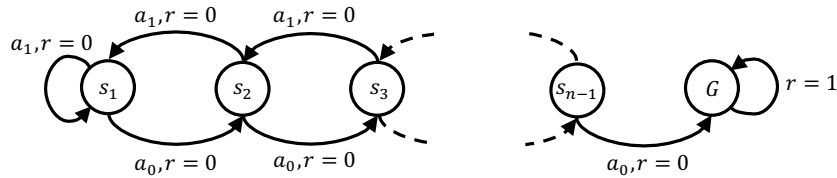


Figure 1: n -state MDP

1.1 Optimal action [5 points]

What is the optimal policy, and does it depend on the value of the discount factor γ ? Explain your answer.

The optimal policy is to always move right at every state, except at the goal state where only one action is possible. This is because only the rightmost point achieves nonzero reward and we lose nothing by moving right every time. This argument holds regardless of the discount factor and so does not depend on it.

1.2 Optimal value function [10 points]

Find the optimal value function for all states s_i and the goal state G .

For goal state G :

$$V^*(G) = 1 + \gamma V^*(G) \implies V^*(G) = \frac{1}{1 - \gamma}$$

For all states $s_i, i \in [n - 1]$: from s_i , the agent takes $(n - i)$ steps to reach G , receiving zero reward until arrival:

$$V^*(s_i) = \gamma^{n-i} V^*(G) = \frac{\gamma^{n-i}}{1 - \gamma}$$

1.3 Adding a constant to each reward [20 points]

Consider adding a constant c to all rewards (i.e. taking any action from states s_i has reward c and any action from the goal state G has reward $1 + c$). Can adding a constant reward c change the optimal policy? Derive the new optimal value function for all states s_i and the goal state G . Explain your answer.

The optimal policy does not change because we still get the maximum long-term reward by getting to the rightmost state and continue lose nothing by moving right every time.

For goal state G :

$$V^*(G) = (1 + c) + \gamma V^*(G) \implies V^*(G) = \frac{1 + c}{1 - \gamma}$$

For non-goal states s_i where $i < n$: from s_i , moving right optimally, the agent receives reward c for $(n - i)$ steps until reaching G :

$$V^*(s_i) = c + \gamma c + \gamma^2 c + \dots + \gamma^{n-i-1} c + \gamma^{n-i} V^*(G) \quad (1)$$

$$= c \sum_{k=0}^{n-i-1} \gamma^k + \gamma^{n-i} \cdot \frac{1 + c}{1 - \gamma} \quad (2)$$

$$= c \cdot \frac{1 - \gamma^{n-i}}{1 - \gamma} + \frac{\gamma^{n-i}(1 + c)}{1 - \gamma} \quad (3)$$

$$= \frac{c(1 - \gamma^{n-i}) + \gamma^{n-i}(1 + c)}{1 - \gamma} = \frac{c + \gamma^{n-i}}{1 - \gamma} \quad (4)$$

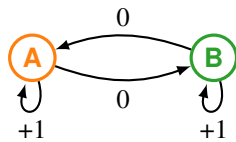
1.4 Scaling all the rewards by a constant [15 points]

After adding a constant c to all rewards now consider scaling all the rewards by a constant a (i.e. $r_{new} = a(c + r_{old})$). Can scaling rewards by a change the optimal policy? Derive the new optimal value function for all states s_i and the goal state G .

Yes, if $a < 0$ then (one) optimal policy is to always move left, except at the leftmost state where you can just stay in place and the rightmost state where you remain stuck. If $a \geq 0$ then we have simply rescaled all the rewards and the policy stays the same so the optimal value function is just $V^*(s_i) = \frac{a}{1-\gamma}(c + \gamma^{n-i})$ and $V^*(G) = \frac{a}{1-\gamma}(c + 1)$. If $a < 0$ then the optimal value function is $V^*(s_i) = ac + \gamma ac + \gamma^2 ac + \dots = \frac{ac}{1-\gamma} \forall i \in [n-1]$ and $V^*(G) = a(c + 1) + \gamma(c + 1) + \gamma^2(c + 1) + \dots = \frac{a(c+1)}{1-\gamma}$ otherwise.

2 Q-learning

Consider the following Markov Decision Process. It has two states s . It has two actions a : move and stay. The state transition is deterministic: “move” moves to the other state, while “stay” stays at the current state. The reward r is 0 for move, 1 for stay. There is a discounting factor $\gamma = 0.8$.



The reinforcement learning agent performs Q-learning. Recall the Q table has entries $Q(s, a)$. The Q table is initialized with all zeros. The agent starts in state $s_1 = A$. In any state s_t , the agent chooses the action a_t according to a behavior policy $a_t = \pi_B(s_t)$. Upon experiencing the next state and reward s_{t+1}, r_t the update is:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') \right).$$

Let the step size parameter $\alpha = 0.5$.

2.1 Greedy [15 points]

Run Q-learning for 200 steps with a deterministic greedy behavior policy: at each state s_t use the best action $a_t \in \arg \max_a Q(s_t, a)$ indicated by the current Q table. If there is a tie, prefer move. Show the Q table at the end.

Define Q table as below: $Q(0,0)$ means the Q for state A, and action “stay”, $Q(0,1)$ means state A and action “move”, etc. Then the final Q table is as below: $([[0., 0.], [0., 0.]])$

2.2 ϵ -greedy [15 points]

Reset and repeat the above, but with an ϵ -greedy behavior policy: at each state s_t , with probability $1 - \epsilon$ choose what the current Q table says is the best action: $\arg \max_a Q(s_t, a)$; Break ties arbitrarily. Otherwise, (with probability ϵ) uniformly chooses between move and stay (move or stay both with 1/2 probability). Use $\epsilon = 0.5$.

Define Q table as below: Q(0,0) means the Q for state A, and action "stay", Q(0,1) means state A and action "move", etc. Then the final Q table is as below:

([[4.99133975607778, 3.99780963857094], [4.99878625277484, 3.99144795447584]])

2.3 Bellman equation [20 points]

Without doing simulation, use Bellman equation to derive the true Q table induced by the MDP.

It is obvious that the optimal policy is that for any state, the optimal action is always "stay".

For bellman equation we have

$$V(0) = 1 + 0.8V(0);$$

$$V(1) = 1 + 0.8V(1);$$

$$Q(0,0) = 1 + 0.8V(0);$$

$$Q(0,1) = 0.8V(1);$$

$$Q(1,0) = 1 + 0.8V(1);$$

$$Q(1,1) = 0.8V(0).$$

So we have that $V(0) = V(1) = 5$, $Q(0,0) = Q(1,0) = 5$, $Q(0,1) = Q(1,1) = 4$.